Sistema de préstamo de equipos de laboratorio

Arquitectura de la solución · ASP.NET Core 10 · tres proyectos y una base de datos compartida

Docker Compose (opción A): red interna, puertos del host 5101–5103 → 8080 y 14330 → 1433. Opción B: los mismos procesos en el equipo, con SQL Server Express.

Navegador

Usuarios por rol

Admin
Mantiene catálogos y usuarios; ve reportes

Docente
Presta, recibe y ve todos los préstamos

Estudiante
Ve equipos disponibles y sus propios préstamos

HTML por HTTP
cookie de sesión

EquiposMVC

Interfaz de operación · http://localhost:5102

Controllers + vistas Razor
Equipo, Categoria, Prestamo, Auth, Home

Services (un HttpClient por servicio)
Auth, Categoria, Equipo y Prestamo

ApiTokenHandler
Agrega Authorization: Bearer a cada petición

Session (2 horas de inactividad)
Guarda token, rol y username en el servidor

Sin acceso a datos
No tiene DbContext ni cadena de conexión

REST + JSON
Bearer JWT

EquiposAPI

Reglas de negocio · http://localhost:5101

JwtBearer + [Authorize(Roles)]
Token de 60 minutos, sin tolerancia de reloj

Controllers REST /api
auth, categorias, equipos, prestamos, usuarios

Servicios
TokenService, PasswordHasher, AutoMapper

EF Core (código primero)
EquiposDbContext y migraciones

Arranque y herramientas
AdminSeeder; Swagger en Development

EF Core
lee y escribe

SQL Server

Base EquiposDb

Categorias
Nombre único

Equipos
FK a Categorias; rowversion

Usuarios
Username único; hash; rol

Prestamos
FK a Equipos y Usuarios

Restricciones
CHECK de Estado y Rol
Un préstamo activo por equipo (índice único filtrado)
FK sin borrado en cascada

Logins
API: sa (Docker) o Windows (local)
Reportes: reportes, solo db_datareader

Inicialización
La API aplica migraciones; db-init crea el login reportes

Enlaces entre aplicaciones;
Reportes pide login otra vez

EquiposReportes

Reportes solo para Admin · http://localhost:5103

ReportesController [Authorize Admin]
Inventario, Vencidos, Top 5 e Historial

CuentaController + cookie
Valida en la API; sin usuarios propios

ReportesService + EF Core (BD primero)
Consultas con Scaffold-DbContext

HTML por HTTP
cookie de auth

POST /api/auth/login
Si el rol no es Admin, se rechaza

EF Core · usuario SQL reportes
solo lectura (db_datareader)

Conexiones

- Petición HTTP
- SQL de lectura y escritura
- SQL de solo lectura
- Navegación entre aplicaciones

Reglas de la arquitectura

Solo EquiposAPI escribe en la base de datos.
EquiposMVC trabaja únicamente a través de la API.
EquiposReportes lee la base directamente y valida el login en la API.

Secretos

Opción A: archivo .env
Opción B: User Secrets
Nunca en appsettings.json